

TASK BRIEF

Project Submission AI Analyzer

with Proctored Live Viva

Build a Python service from scratch that takes a student's project submission through to a mentor-ready evaluation — from analyzing the codebase, to generating interview questions, to running and monitoring the live viva itself.

1. Overview

Build a Python service from scratch that analyzes a student's project submission (ZIP file), prepares a mentor-led viva around it, and monitors that viva as it happens. Given only project metadata, a ZIP, and a skill catalog, the service should:

1. Extract and analyze the codebase
2. Suggest relevant skills from the provided catalog
3. Generate interview questions per skill
4. Run a proctored live viva session while the student answers those questions
5. Return a single structured evaluation package (JSON) combining the technical evaluation and the viva's integrity report

2. Problem

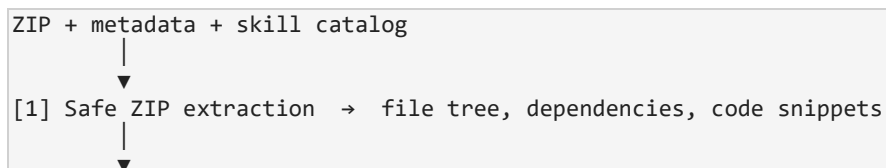
Universities assess experiential projects (internships, capstones, mini-apps). Mentors need help preparing viva questions that reflect what the student actually built — not generic textbook questions.

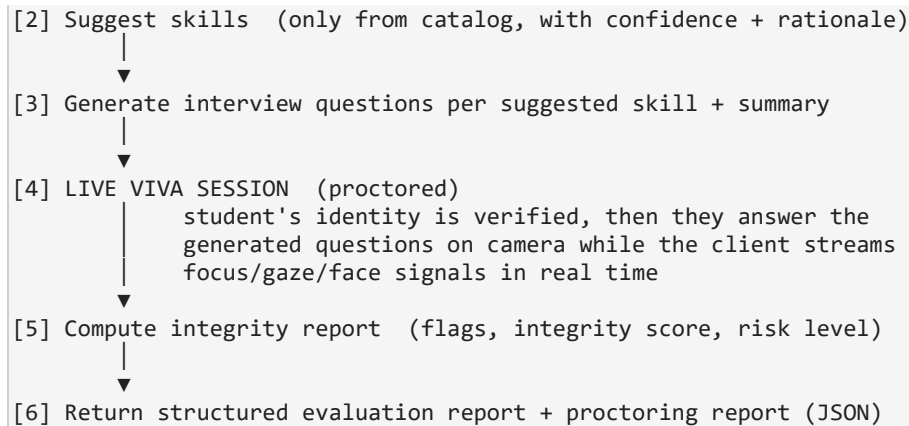
The viva itself also needs a basic integrity layer: mentors need confidence that the student answering is the student who submitted the project, that they stayed on-screen and attentive, and that they weren't quietly referencing outside help during questioning.

Input: ZIP file + project metadata + skill catalog (JSON we provide)

Output: ranked skills + skill-wise questions + outcome evaluation report + live-session proctoring/integrity report

3. Required Flow





4. Submission Analysis API

One endpoint:

POST /analyze-submission

Input (multipart form)

Field	Type	Required
project_title	string	yes
project_description	string	no
project_outcomes	string	yes
zip_file	file	yes
questions_per_skill	int	no (default: 2)

Skill catalog (provided, stored inside the app as JSON — no database queries)

```
[
  { "skill_id": "uuid-1", "skill_name": "Python" },
  { "skill_id": "uuid-2", "skill_name": "FastAPI" },
  { "skill_id": "uuid-3", "skill_name": "PostgreSQL" }
]
```

Rules

- Only suggest skills that exist in the catalog
- Every suggestion must include confidence (0–1) and rationale
- Skills may be added to the catalog if needed

Summary requirements

- Compare ZIP evidence against each outcome in project_outcomes (split on newlines/numbered bullets)
- status per outcome: met | partial | not_met | not_verifiable
- evidence must cite real files, modules, or patterns from the ZIP — not generic claims
- overall_alignment: strong | partial | weak, with an optional alignment_score (0–1)

- narrative: 2–4 plain-English sentences for the mentor, not bullet-only

5. Live Viva Proctoring

During the viva, the student answers the generated questions on camera through the same session. The service must watch for basic integrity signals in real time and fold the result into the final report — it is not a separate deliverable, it is one more stage of the same pipeline.

Design principle: signals, not surveillance

No raw video or audio is stored or transmitted to the backend. Face/eye landmark extraction happens client-side (e.g. in-browser via a JS face-landmark model); the client only ever sends lightweight, timestamped events. This keeps the service privacy-respecting and keeps bandwidth/compute costs near zero.

New endpoints

POST /viva-session/start	→ start a session tied to an analyze-submission result
POST /viva-session/event	→ client posts one integrity event (see schema below)
POST /viva-session/end	→ close the session, compute the integrity report

Session lifecycle events

event_type	When it fires
interview_started	The viva session begins and monitoring goes live
id_verified	The student's face/ID matches the submission record
id_failed	Identity check fails — face/ID does not match, or no ID presented
snapshot_captured	A periodic reference snapshot is captured during the session

Integrity flag events

event_type	Detection basis
gaze_off_screen	eye/gaze vector off-screen beyond a threshold duration
multiple_faces_detected	more than one face detected simultaneously
face_not_detected	no face detected in the landmark stream for > N seconds
tab_switched	Page Visibility API — document.hidden fires
fullscreen_exited	Fullscreen API — fullscreenchange fires
paste_attempted	a clipboard paste event fires during an active question
screenshot_detected	a screen-capture / print-screen attempt is detected

Event schema (client → server)

```
{  
  "session_id": "sess-abc123",  
  "event_type": "gaze_off_screen",  
  "timestamp": "2026-07-05T10:12:03Z",  
}
```

```

    "duration_ms": 4200,
    "confidence": 0.81
  }

```

Proctoring rules

- A session always opens with `interview_started` and an identity check that resolves to `id_verified` or `id_failed` before questioning begins
- Every flag must include a type, timestamp, and severity (low | medium | high)
- Severity thresholds (e.g. `gaze_off_screen > 3s` = low, `> 8s` = medium) must be configurable, not hard-coded
- A session must always produce an `integrity_score` (0–1) and `risk_level` (low | medium | high), even with zero flags
- The student must see a one-time consent notice before the session starts explaining what is monitored
- If the client stops sending events entirely (connection lost, camera denied), this itself must be flagged — not treated as a clean session

6. Combined Output (JSON)

The final response merges both modules into a single evaluation package:

```

{
  "project_title": "...",
  "suggested_skills": [
    { "skill_id": "uuid-1", "skill_name": "Python",
      "confidence": 0.92, "rationale": "Python used in API and service layers" }
  ],
  "evaluation_report": {
    "skills": [ { "skill_name": "Python", "questions": [ /* conceptual + codebase_specific */
    ] } ],
    "summary": {
      "overall_alignment": "partial",
      "alignment_score": 0.72,
      "narrative": "...",
      "outcome_evaluation": [ /* per stated outcome: status, evidence, gap */ ],
      "strengths": [ "..." ],
      "gaps": [ "..." ]
    }
  },
  "proctoring_report": {
    "session_id": "sess-abc123",
    "id_check": "id_verified",
    "integrity_score": 0.86,
    "risk_level": "low",
    "flag_summary": { "gaze_off_screen": 3, "tab_switched": 1,
      "face_not_detected": 0, "multiple_faces_detected": 0 },
    "flags": [
      { "type": "gaze_off_screen", "timestamp": "2026-07-05T10:12:03Z",
        "duration_ms": 4200, "severity": "low" }
    ],
    "narrative": "The student remained on-screen for the large majority of the
      session; a small number of brief glances away were recorded
      with no evidence of external assistance."
  },
  "metadata": { "files_analyzed": 12, "extraction_time_ms": 340, "model_tokens_used": 2100 },

```

```
"processing_time_ms": 4200
}
```

7. Technical Requirements

Item	Requirement
Language	Python 3.11+
Framework	FastAPI
LLM	Use your own AI key
Data	No database — skill catalog is a saved file; proctoring events kept in-memory or a lightweight local store for the session's lifetime
Client-side capture	Browser getUserMedia + a JS face-landmark model (e.g. face-api.js / MediaPipe FaceMesh in-browser) — raw video never leaves the client
Proctoring transport	Lightweight event objects only (see schema in §5) — no frames, no audio
Questions	Per skill: at least 1 conceptual + 1 codebase_specific (must cite real file paths or symbols from the ZIP)
Errors	Clear HTTP responses for bad ZIP, empty project, invalid catalog, LLM failures, and dropped proctoring connections
Privacy	No raw video/audio storage; explicit on-screen consent before a session starts
Config	.env.example for API keys and proctoring thresholds — no secrets in submission

8. What We Provide

- Shared Git repository (access will be given to you)
- Sample skill catalog JSON
- Optional: 1–2 example ZIP submissions for testing

Everything else — project structure, ZIP parsing, prompts, API, proctoring event handling, tests — you build from scratch.

9. Deliverables

6. Complete runnable project — install deps + one command to start the server
7. README — setup, env vars, folder structure, how to run tests, one example API call for /analyze-submission, one example flow for /viva-session/*, and demo video link
8. Your own test ZIP — small project you zipped for demo (5–15 source files)
9. 2+ unit tests (optional) — e.g. path-traversal ZIP rejected; proctoring event schema validated; output matches your Pydantic schema
10. Short write-up (optional, ½ page) in README or NOTES.md — include one paragraph on how you approached privacy for the proctoring signals

10. Submission

1. Git branch (required)

Push your code to the shared repository we provide. Do not commit directly to main or master.

Branch naming convention:

```
<college-id>-<full-name-in-lowercase-with-hyphens>
```

Rule	Good	Bad
College ID first, then name	21cs0123-arun-kumar	arun-kumar-21cs0123
All lowercase	22it0456-priya-sharma	22IT0456-Priya-Sharma
Hyphens between words	21ec0789-r-venkatesh	21ec0789_r_venkatesh
No spaces or special characters	20cs1001-john-doe	20cs1001 john doe

Steps

```
git clone <shared-repo-url>
cd <repo-name>
git checkout -b 21cs0123-arun-kumar
# ... add your code ...
git add .
git commit -m "Add project submission analyzer with viva proctoring"
git push -u origin 21cs0123-arun-kumar
```

Do not commit: API keys, .env files, or secrets. Use .env.example only.

2. Working demo video (required)

Record a 5–10 minute video showing the project working end to end. Must show:

11. Starting the server locally
12. One successful POST /analyze-submission call with a real ZIP and skill catalog
13. A live /viva-session run — including at least one deliberately triggered proctoring flag (e.g. looking away, switching tabs)
14. The final JSON response — suggested skills, questions, outcome evaluation, and the proctoring report
15. Brief walkthrough of your code structure and one design decision

Item	Requirement
Format	Share MP4 in WhatsApp group or a shareable link (Google Drive / OneDrive — view access for reviewers). Slideshows or code-only walkthroughs without a live demo will not be accepted.

11. Sample Test ZIP Ideas

Candidates create their own small project, zip it, and test against it:

- FastAPI CRUD app

- React + Express todo app
- Python data script with requirements.txt

Avoid empty ZIPs or single-file homework.